UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables

Carrera de Computación

CUADERNO DE SÍNTESIS

UNIDAD 1

Análisis de lógica combinacional con diseño clásico

Dato académico	Información
Asignatura	Diseño de Circuitos
Estudiante	Alexander Ventura Guaman Puli
Paralelo	Segundo ciclo – paralelo O
Docente	Ing. Luis Darío Sinche Cueva
Período académico	Septiembre 2026 – marzo 2027
Fecha de entrega	11 de noviembre de 2026

Loja – Ecuador

Presentación

Este cuaderno reúne y organiza los contenidos fundamentales de la Unidad 1 de Diseño de Circuitos. Su propósito es sintetizar los conceptos, las reglas, las expresiones booleanas y los procedimientos utilizados para analizar y diseñar circuitos lógicos combinacionales.

La síntesis se organiza en cuatro semanas: sistemas numéricos y compuertas lógicas; álgebra de Boole y formas canónicas; mapas de Karnaugh y compuertas universales; y bloques combinacionales aritméticos. En cada semana se incluyen definiciones breves, fórmulas, ejemplos y observaciones para el estudio.

Objetivo de aprendizaje

Comprender el procedimiento clásico de diseño combinacional: interpretar un requerimiento, identificar entradas y salidas, construir la tabla de verdad, obtener y simplificar la función lógica y finalmente implementarla mediante compuertas.

Mapa general de la unidad

Semana	Tema	Herramienta principal
1	Sistemas numéricos, códigos y compuertas lógicas	Conversiones, representación binaria y tablas de verdad
2	Álgebra de Boole y formas canónicas	Leyes booleanas, minitérminos, maxitérminos y simplificación
3	Mapas de Karnaugh y compuertas universales	Minimización gráfica, condiciones indiferentes y NAND/NOR
4	Bloques combinacionales aritméticos	Semisumador, sumador completo, suma/resta y comparador

Idea integradora Cada tema aporta una etapa para transformar una necesidad expresada en palabras en un circuito combinacional verificable.

SEMANA 1. Sistemas numéricos, códigos y compuertas lógicas

Idea principal: los sistemas digitales representan información mediante dos estados lógicos, usualmente 0 y 1. Los números, caracteres y operaciones se codifican en bits; las compuertas lógicas procesan esos bits.

1.1 Sistemas de numeración

Sistema	Base	Símbolos	Equivalencia útil
Binario	2	0 y 1	Cada posición representa una potencia de 2.
Octal	8	0 al 7	Cada dígito octal equivale a 3 bits.
Decimal	10	0 al 9	Sistema de uso cotidiano.
Hexadecimal	16	0 al 9 y A al F	Cada dígito hexadecimal equivale a 4 bits.

1.2 Conversión de decimal a binario

Procedimiento: dividir sucesivamente el número decimal entre 2, registrar cada residuo y leer los residuos desde el último hasta el primero.

División	Cociente	Residuo
37 ÷ 2	18	1
18 ÷ 2	9	0

División	Cociente	Residuo
9 ÷ 2	4	1
4 ÷ 2	2	0
2 ÷ 2	1	0
1 ÷ 2	0	1

$$37_{10} = 100101_2$$

Comprobación: $100101_2 = 1\cdot2^5 + 0\cdot2^4 + 0\cdot2^3 + 1\cdot2^2 + 0\cdot2^1 + 1\cdot2^0 = 32 + 4 + 1 = 37$.

1.3 Conversiones a octal y hexadecimal

Para pasar de binario a octal, se agrupan los bits de tres en tres desde la derecha. Para pasar a hexadecimal, se agrupan de cuatro en cuatro. Si falta algún bit a la izquierda, se completa con ceros.

Representación	Resultado
37 decimal en binario	100101_2
$100\ 101_2$ agrupado en ternas	$4\ 5 \rightarrow 45_8$
$0010\ 0101_2$ agrupado en cuartetos	$2\ 5 \rightarrow 25_{16}$

1.4 Números con signo: complemento a dos

El complemento a dos permite representar números negativos. Para obtener −N en una cantidad fija de bits: escribir N en binario, invertir todos los bits y sumar 1.

1. Representar +5 en 8 bits: 00000101.
2. Invertir los bits: 11111010.
3. Sumar 1: 11111011.

$$-5 \text{ en 8 bits} = 11111011_2$$

En complemento a dos con n bits, el rango es $-2^{(n-1)}$ a $+2^{(n-1)}-1$. Para 8 bits: −128 a +127.

Corrección importante 11111010 es el resultado de invertir los bits; todavía no es −5. El resultado final, después de sumar 1, es 11111011.

1.5 Códigos digitales

Código	Descripción	Ejemplo
BCD	Cada dígito decimal se codifica por separado usando 4 bits.	59 → 0101 1001
Gray	Entre dos valores consecutivos cambia un solo bit.	00, 01, 11, 10
ASCII	Código para representar caracteres. ASCII estándar utiliza 7 bits.	$A = 65_{10} = 01000001_2 = 0x41$

En BCD, 59 se representa como 0101 1001. No debe confundirse con la conversión binaria directa de 59, que es 111011_2.

1.6 Compuertas lógicas

Una compuerta lógica ejecuta una operación booleana sobre una o más entradas y produce una salida. La siguiente tabla resume las compuertas básicas de dos entradas.

A	B	AND (AB)	OR (A+B)	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1

A	B	AND (AB)	OR (A+B)	NAND	NOR	XOR	XNOR
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

La compuerta NOT invierte el valor: si A=0, A'=1; si A=1, A'=0.

1.7 Circuitos integrados de la familia 74LS

Circuito integrado	Función habitual
7400	Cuatro compuertas NAND de 2 entradas
7402	Cuatro compuertas NOR de 2 entradas
7404	Seis inversores NOT
7408	Cuatro compuertas AND de 2 entradas
7432	Cuatro compuertas OR de 2 entradas
7486	Cuatro compuertas XOR de 2 entradas
7485	Comparador de magnitud de 4 bits
74283 / 74LS283	Sumador binario completo de 4 bits

Para recordar El número de compuertas dentro de un CI y su distribución de pines deben verificarse en la hoja de datos del fabricante. También se debe comprobar alimentación y niveles lógicos.

Síntesis de la semana 1

Los sistemas numéricos permiten representar cantidades; los códigos representan información; y las compuertas realizan operaciones lógicas. Comprender las conversiones y las tablas de verdad es la base para analizar cualquier circuito digital.

SEMANA 2. Álgebra de Boole y formas canónicas

Idea principal: una función booleana describe el comportamiento lógico de un circuito. Las leyes del álgebra de Boole permiten transformar y simplificar esa función sin cambiar su comportamiento.

2.1 Leyes fundamentales del álgebra de Boole

Ley	Operación AND	Operación OR
Identidad	A·1 = A	A + 0 = A
Elemento nulo	A·0 = 0	A + 1 = 1
Idempotencia	A·A = A	A + A = A
Complemento	A·A' = 0	A + A' = 1
Conmutativa	AB = BA	A+B = B+A
Asociativa	(AB)C = A(BC)	(A+B)+C = A+(B+C)
Distributiva	A(B+C)=AB+AC	A+BC=(A+B)(A+C)
Absorción	A(A+B)=A	A+AB=A
Simplificación	A(A'+B)=AB	A+A'B=A+B

2.2 Teoremas de De Morgan

$$(A \cdot B)' = A' + B'$$

$$(A + B)' = A' \cdot B'$$

Los teoremas de De Morgan permiten transformar expresiones negadas y son especialmente útiles para implementar funciones con compuertas NAND y NOR.

2.3 Formas canónicas: SOP y POS

- Minitérmino: producto que contiene todas las variables, directas o negadas. Se asocia a una fila de la tabla de verdad donde F=1.
- SOP (suma de productos): suma lógica de los minitérminos correspondientes a las filas donde F=1.
- Maxitérmino: suma que contiene todas las variables, directas o negadas. Se asocia a una fila donde F=0.
- POS (producto de sumas): producto lógico de los maxitérminos correspondientes a las filas donde F=0.

Para construir un minitérmino: una variable con valor 1 aparece directa y una con valor 0 aparece negada. Para construir un maxitérmino se usa la regla inversa: valor 0 directa y valor 1 negada.

2.4 Ejercicio resuelto: alarma de tres sensores

Requerimiento: una sirena F se activa cuando al menos dos de los tres sensores A, B y C están encendidos.

A	B	C	F	Interpretación
0	0	0	0	Ningún sensor activo
0	0	1	0	Un sensor activo
0	1	0	0	Un sensor activo
0	1	1	1	Dos sensores activos
1	0	0	0	Un sensor activo
1	0	1	1	Dos sensores activos
1	1	0	1	Dos sensores activos
1	1	1	1	Tres sensores activos

La función vale 1 en los minitérminos 3, 5, 6 y 7.

$$F(A,B,C) = \Sigma m(3,5,6,7)$$

$$F = A'BC + AB'C + ABC' + ABC$$

Al agrupar los términos que difieren en una sola variable, se obtiene la expresión simplificada:

$$F = AB + AC + BC$$

Interpretación: basta con que cualquier pareja de sensores esté activa para que la sirena se encienda.

Implementación	Elementos lógicos	Cantidad de compuertas
Forma canónica directa	3 NOT + 4 AND de 3 entradas + 1 OR de 4 entradas	8
Forma simplificada	3 AND de 2 entradas + 1 OR de 3 entradas	4

Observación La cantidad anterior cuenta bloques lógicos ideales. El número real de circuitos integrados depende de las entradas disponibles en cada CI, del encapsulado y de cómo se conecte el circuito.

Síntesis de la semana 2

El álgebra de Boole permite expresar el comportamiento de un circuito y simplificarlo. Las formas SOP y POS ofrecen representaciones canónicas que parten directamente de la tabla de verdad.

SEMANA 3. Mapas de Karnaugh y compuertas universales

Idea principal: el mapa de Karnaugh organiza los valores de una función en un orden Gray para identificar agrupaciones de unos o ceros. Estas agrupaciones permiten obtener expresiones más sencillas.

3.1 Reglas de agrupación

- El número de celdas de cada grupo debe ser una potencia de 2: 1, 2, 4, 8 o 16.
- Los grupos deben ser rectangulares y contener celdas adyacentes.
- Se deben buscar grupos tan grandes como sea posible para eliminar variables.
- Los bordes opuestos del mapa también son adyacentes; por eso puede haber agrupaciones que atraviesen los extremos.
- Una celda puede pertenecer a más de un grupo si eso ayuda a cubrir todos los unos.
- Las condiciones indiferentes X pueden tomarse como 0 o como 1, según convenga. No es obligatorio utilizarlas.

3.2 Orden Gray para un mapa de cuatro variables

En un mapa de 4 variables, las filas y las columnas se ordenan 00, 01, 11, 10. Este orden asegura que dos posiciones consecutivas cambien en un solo bit.

AB \ CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

3.3 Ejercicio resuelto con condiciones indiferentes

$$F(A,B,C,D) = \Sigma m(1,3,7,11,15) + d(0,2,5)$$

Se identifican los unos en los minitérminos 1, 3, 7, 11 y 15; las condiciones indiferentes están en 0, 2 y 5.

AB \ CD	00	01	11	10
00	X	1	1	X
01	0	X	1	0
11	0	0	1	0
10	0	0	1	0

Agrupaciones:

1. Grupo 1: m1, m3, d5 y m7. En estas celdas A=0 y D=1 permanecen constantes; se obtiene A'D.
2. Grupo 2: m3, m7, m11 y m15. En estas celdas C=1 y D=1 permanecen constantes; se obtiene CD.

$$F = A'D + CD = D(A' + C)$$

Verificación La expresión cubre todos los minitérminos requeridos. La condición indiferente m5 se utiliza para ampliar el primer grupo.

3.4 Universalidad de NAND y NOR

NAND y NOR son compuertas universales porque cualquier función lógica puede implementarse utilizando exclusivamente compuertas de uno de estos tipos.

Función	Implementación con NAND	Implementación con NOR
NOT	A' = (A·A)'	A' = (A+A)'

Función	Implementación con NAND	Implementación con NOR
AND	AB = ((AB)')'	Se puede formar a partir de NOR y negaciones
OR	A+B = (A'·B')'	Se puede formar a partir de NOR y negaciones

Una función SOP puede implementarse mediante una estructura NAND–NAND. Una función POS puede implementarse mediante una estructura NOR–NOR.

3.5 Implementación NAND del ejercicio

$$F = A'D + CD$$

$$F = ((A'D)' \cdot (CD)')'$$

La implementación requiere generar A' con una NAND usada como inversor, obtener las negaciones de los productos A'D y CD con dos NAND y combinar esas salidas con una NAND final. En total: cuatro compuertas NAND.

Implementación	Compuertas aproximadas	CI típico
AND–OR con inversor	1 NOT + 2 AND + 1 OR	7404, 7408 y 7432
Solo NAND	4 NAND	7400

Síntesis de la semana 3

Karnaugh permite minimizar funciones de forma visual. Las condiciones indiferentes ofrecen flexibilidad en las agrupaciones. NAND y NOR facilitan implementar circuitos utilizando un solo tipo de compuerta.

SEMANA 4. Bloques combinacionales aritméticos

Idea principal: los circuitos aritméticos realizan operaciones binarias. Sus salidas dependen de las entradas actuales y, en los sumadores, del acarreo que llega desde la etapa anterior.

4.1 Semisumador (Half Adder)

El semisumador suma dos bits A y B. Produce una salida de suma S y un acarreo de salida Cout. No tiene entrada de acarreo.

$$S = A \oplus B$$

$$Cout = A \cdot B$$

A	B	S	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

4.2 Sumador completo (Full Adder)

El sumador completo suma tres bits: A, B y Cin (acarreo de entrada). Produce la suma S y el acarreo de salida Cout.

$$S = A \oplus B \oplus Cin$$

$$Cout = AB + A \cdot Cin + B \cdot Cin$$

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4.3 Sumador paralelo de 4 bits

Un sumador paralelo de 4 bits conecta cuatro sumadores completos en cadena. Cada etapa suma un par de bits y recibe el acarreo de la etapa anterior. El acarreo de salida de una etapa se conecta al Cin de la siguiente.

Nombre habitual Esta arquitectura también se conoce como sumador con acarreo en cascada o ripple-carry adder. El acarreo debe propagarse entre etapas, por lo que el retardo aumenta con el número de bits.

4.4 Resta mediante complemento a dos

La resta binaria puede realizarse utilizando un sumador: se invierten los bits del sustraendo B y se suma 1.

$$A - B = A + B' + 1$$

En un circuito sumador/restador, las compuertas XOR controladas por una señal de modo M permiten seleccionar la operación: con M=0 se suma B sin invertir; con M=1 se invierte B y se introduce Cin=1.

4.5 Desbordamiento (Overflow)

El desbordamiento ocurre en operaciones con números con signo cuando el resultado matemático no puede representarse con la cantidad de bits disponible.

$$V = \text{acarreo de entrada al bit más significativo} \oplus \text{acarreo de salida del bit más significativo}$$

En una suma de complemento a dos, también puede reconocerse el overflow cuando se suman dos números del mismo signo y el resultado aparece con signo diferente.

No confundir El acarreo de salida (Cout) es importante en operaciones sin signo. El overflow (V) indica que el resultado con signo no cabe en el rango representable. No significan lo mismo.

4.6 Ejemplo: suma de 5 + 3 con 4 bits

$$5 = 0101_2 \quad 3 = 0011_2$$

$$0101 + 0011 = 1000$$

El resultado binario de cuatro bits es 1000. Como número sin signo representa 8 y es válido. Sin embargo, en complemento a dos de 4 bits el rango es −8 a +7; por tanto, 1000 representa −8 y se produjo overflow al intentar representar +8.

Operación	Cálculo en 4 bits	Resultado de 4 bits	Interpretación
5 + 3	0101 + 0011	1000	Overflow con signo; 8 no cabe en −8 a +7.
7 − 3	0111 + 1100 + 1	0100	Resultado +4; acarreo final 1.
3 − 7	0011 + 1000 + 1	1100	Resultado −4 en complemento a dos; acarreo final 0.

En las restas mostradas, el acarreo final se presenta según la convención habitual del sumador: para una resta sin signo, Cout=1 indica que no hubo préstamo y Cout=0 indica que sí hubo préstamo. Para interpretar resultados con signo también se debe revisar el overflow.

4.7 Comparadores de magnitud

Un comparador combinacional determina la relación entre dos números binarios A y B. Sus salidas típicas indican si A>B, A=B o A<B.

Salida	Significado
A>B	A es mayor que B.
A=B	A y B tienen el mismo valor.
A<B	A es menor que B.

Los comparadores pueden construirse con compuertas lógicas o utilizar circuitos integrados específicos, como el 7485, que compara dos números de 4 bits y dispone de entradas para encadenar comparadores.

Síntesis de la semana 4

Los semisumadores y sumadores completos son bloques básicos para construir sumadores de varios bits. La resta se puede convertir en suma mediante complemento a dos. Para interpretar correctamente los resultados es necesario distinguir entre acarreo y desbordamiento.

Aplicación integradora de la unidad

Un ejemplo de aplicación es el diseño de una alarma digital para un sistema de seguridad con tres sensores. Primero se identifican las entradas A, B y C; luego se establece que la salida debe activarse cuando al menos dos sensores detecten una condición. Se construye la tabla de verdad, se obtiene la función canónica y se simplifica con álgebra de Boole o Karnaugh.

$$F = AB + AC + BC$$

Finalmente, la función se implementa mediante compuertas AND y OR o mediante una red equivalente de compuertas NAND. Este proceso integra la representación binaria, la tabla de verdad, el álgebra booleana, la minimización y la implementación física.

Conclusiones personales

- Los sistemas numéricos y los códigos permiten representar información dentro de los sistemas digitales.
- Las compuertas lógicas y el álgebra de Boole permiten describir y analizar el comportamiento de un circuito.
- Las formas canónicas y los mapas de Karnaugh ayudan a obtener expresiones lógicas equivalentes y más sencillas.
- Las compuertas NAND y NOR permiten implementar cualquier función lógica y pueden reducir la variedad de circuitos integrados necesarios.
- Los bloques aritméticos permiten realizar operaciones binarias; para interpretar resultados con signo es indispensable reconocer el desbordamiento.

Registro de dudas pendientes

N.º	Duda registrada	Dónde se resolverá
1	Cuando un mapa de Karnaugh tiene varias agrupaciones mínimas equivalentes, ¿cómo elijo la que utiliza menos compuertas o menos entradas?	Tutoría / taller
2	¿Una condición indiferente debe tratarse siempre como 1 si agranda un grupo, aunque cambie el costo de otra parte del circuito?	Tutoría académica
3	¿Cómo se estima el retardo real de un sumador paralelo a partir de la hoja de datos de un CI, por ejemplo el 74LS283?	Hoja de datos / práctica
4	¿En qué casos conviene utilizar POS (NOR–NOR) frente a SOP (NAND–NAND)?	Tutoría académica
5	¿Cómo afecta la bandera de overflow al flujo de instrucciones cuando se trabaja con números sin signo frente a números con signo?	Próxima unidad – ALU

Glosario breve

Término	Significado
Bit	Unidad mínima de información digital; puede valer 0 o 1.
Acarreo (carry)	Bit que se genera y pasa a la siguiente posición en una suma.
Complemento a dos	Método de representación de enteros con signo y de realización de restas.
Minitérmino	Producto de todas las variables que representa una fila con salida 1.

Término	Significado
Maxitérmino	Suma de todas las variables que representa una fila con salida 0.
Condición indiferente	Combinación cuyo resultado puede elegirse como 0 o 1 para facilitar la simplificación.
Overflow	Desbordamiento al representar un resultado con signo.
CI	Circuito integrado.

Recomendaciones para estudiar

1. Practicar conversiones entre decimal, binario, octal y hexadecimal.
2. Memorizar las tablas de verdad de las compuertas y las leyes básicas de Boole.
3. Resolver funciones desde una tabla de verdad y comprobar el resultado con Karnaugh.
4. Dibujar las implementaciones equivalentes con NAND y NOR.
5. Practicar sumas y restas en complemento a dos, diferenciando acarreo y overflow.

Fin del cuaderno de síntesis.